

Vector Search and Approximate Nearest Neighbors

pig7selene

September 5, 2026

Abstract

A dense retriever is useful only if its stored vectors can be searched within the latency and memory budget of a RAG system. This chapter starts from exact vector ranking, then develops Approximate Nearest Neighbor (ANN) indexing as a controlled reduction in candidate-search work. It explains Inverted File indexing, Product Quantization, and Hierarchical Navigable Small World graphs; separates ANN recall from retrieval relevance; and traces the consequences of index construction, compression, filters, hardware, and corpus mutation. The central lesson is that an ANN index changes candidate generation, not the meaning of the embedding model or the correctness of the final answer.

1 Introduction

Chapter 27 defined a dense retriever by a query embedding e_q and a collection of compatible document embeddings. Once those representations exist, retrieval must answer a system's question: which stored vectors are closest to e_q under the model's declared score, and how quickly can the answer be produced? For a small collection, the direct answer is straightforward. Score every vector, rank the results, and return the best k . At corpus scale, reading and comparing every vector can dominate retrieval latency and memory bandwidth.

Vector search provides data structures and search procedures for this problem. It does not replace the query encoder, document encoder, similarity function, or relevance definition from Chapter 27. Instead, it changes how the candidate set is found. This distinction is essential for diagnosis. A weak answer may originate in an embedding model that placed the right document far away, an approximate index that failed to return a nearby vector, a metadata filter that excluded an eligible item, or a later ranking and generation stage. Those failures require different repairs.

This chapter studies vector-only candidate generation. It does not derive chunking, lexical retrieval, hybrid search, Cross-Encoder reranking, query rewriting, or RAG evaluation. These later choices consume the candidate interface established here.

2 Exact Nearest-Neighbor Search

Let $E = \{e_1, \dots, e_N\}$ be the stored embedding vectors, each in R^r , and let $s(e_q, e_i)$ be a score for which larger values are better. The exact top- k set is

$$E_k^{\text{exact}(q)} = \text{TopK}_{e_i \in E} s(e_q, e_i). \quad (1)$$

Brute-force search computes $s(e_q, e_i)$ for every i , selects the best results, and returns their associated retrieval-unit IDs. For an inner product, the score work is on the order of Nr scalar multiply-add operations per query, followed by top- k selection. The linear scan is exact with respect to the declared vectors, metric, filters, and arithmetic convention. It is therefore the right baseline for measuring index behavior.

Exact does not mean universally expensive. A small corpus can be scanned directly; batches of queries can make dense matrix multiplication highly efficient; and a GPU can evaluate many scores in parallel when the vectors fit the relevant memory hierarchy. Exact search is also useful for offline evaluation and index regression tests. Its limitation is scaling: at fixed dimension, every additional vector adds

work and memory traffic to every query. A system that must search a large corpus one request at a time often needs to avoid inspecting most of E .

3 Approximate Nearest Neighbors and Recall

Approximate Nearest Neighbor (ANN) search uses an index to examine a selected subset or an approximate representation of the corpus. The goal is not to change the semantic score definition, but to find high-scoring candidates without exhaustively evaluating every stored vector. The trade-off is explicit: reduced candidate-search work may cause the index to miss an item that exact search would have returned.

For a query with an exact vector-neighbor set $E_k^{\text{exact}(q)}$ and an approximate result set $E_k^{\text{ann}(q)}$, a common index metric is ANN Recall@k:

$$\text{Recall}_{\text{ANN}} @k(q) = \frac{|E_k^{\text{ann}(q)} \cap E_k^{\text{exact}(q)}|}{k}. \quad (2)$$

This metric compares an index with exhaustive vector search under the same embedding space. It is not the same as retrieval-task Recall@k from Chapter 26, which asks whether relevant evidence appears in the result set. An ANN index can have high recall against exact neighbors even when the embedding model retrieves irrelevant passages. Conversely, a lucky approximate miss can occasionally surface a task-relevant item that exact vector ranking would not have chosen. In production, both index recall and task relevance matter; neither substitutes for the other.

ANN methods expose search controls that trade query latency against candidate coverage. More work at query time usually improves ANN recall, but consumes compute, bandwidth, and sometimes queueing capacity. Some indexes also trade greater build time or memory for faster search. There is no configuration that is optimal independently of the corpus, score function, update rate, hardware, workload concurrency, and required quality floor.

4 Inverted File Indexes

An **Inverted File** (IVF) index partitions the vector space into coarse regions. Let $c_1, \dots, c_M$ be coarse centroids learned from representative vectors. Each stored vector receives an assignment

$$a(i) = \arg \min_{m \in \{1, \dots, M\}} \delta(e_i, c_m), \quad L_m = \{i : a(i) = m\}, \quad (3)$$

where δ is the coarse distance convention and L_m is the inverted list for centroid c_m . The phrase **inverted file** refers here to lists of vector IDs associated with coarse cells; it is not the term-posting structure used by a lexical search engine.

At query time, the system first ranks the centroids, selects a small set $P_{p(q)}$ of p promising cells, and scores only vectors in the corresponding lists:

$$C_{\text{IVF}(q)} = \cup_{m \in P_{p(q)}} L_m. \quad (4)$$

It then ranks $C_{\text{IVF}(q)}$ by the original or an approximate vector score. The number of probed regions, represented by p , is a key search control. Probing more clusters enlarges the candidate set, tending to improve ANN recall while increasing query work. Probing too few clusters can miss the correct neighborhood when a query lies near a coarse boundary or the centroid partition is poorly aligned with the embedding distribution.

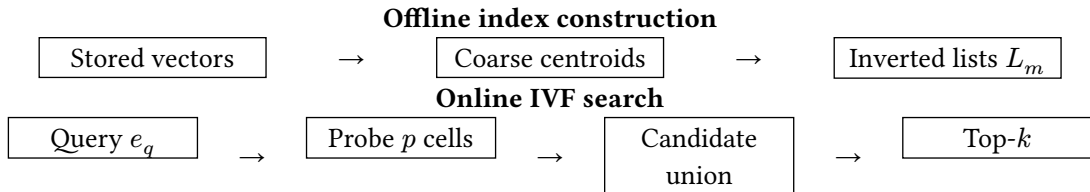


Figure 1: IVF replaces a full corpus scan with centroid selection followed by search inside selected inverted lists. The chosen probe budget determines the candidate-recall versus query-cost trade-off.

5 Product Quantization and Vector Compression

Product Quantization (PQ) compresses stored retrieval vectors. It divides a vector into m subvectors,

$$e = [e^1, \dots, e^m], \quad (5)$$

and assigns each subvector to a codeword from a small subspace-specific codebook. If $\mathcal{C}_j = \{c_{j,1}, \dots, c_{j,K}\}$ is the codebook for block j , the stored code can be written as

$$z_{j(e)} = \arg \min_{\ell \in \{1, \dots, K\}} \|e^j - c_{j,\ell}\|_2. \quad (6)$$

Instead of retaining every full-precision component, the index stores the m code indices $z_1(e), \dots, z_{m(e)}$ and reconstructs or estimates comparisons from the selected codewords. With K codewords per block, the code payload is roughly $m \log_2 K$ bits per vector before IDs and auxiliary structures. PQ therefore trades distance-estimation accuracy for a substantial reduction in vector storage and memory traffic. Product Quantization was introduced for approximate nearest-neighbor search precisely as a compact representation whose distance computations can be organized efficiently [1].

This is different from model quantization in Chapter 21. Model quantization changes how neural-network weights or activations are represented during inference. PQ here compresses the **stored retrieval vectors** and affects the index’s candidate-ranking approximation. The two techniques can coexist, but their numerical errors and quality evaluations belong to different parts of a RAG system. IVF and PQ combine naturally. IVF first limits candidate generation to selected coarse lists; PQ reduces the cost and memory of representing and comparing the vectors inside those lists. The combined index may be much more scalable than a flat full-precision scan, but it introduces two sources of approximation: a relevant vector can be excluded by coarse-cell selection or misordered by compressed distance estimates. Increasing the probe budget, using larger codebooks or more subspaces, or retaining selected full-precision vectors can improve quality at a memory or latency cost.

6 Graph-Based Search and HNSW

Graph-based indexes represent stored vectors as nodes joined by selected proximity edges. In a **Hierarchical Navigable Small World** (HNSW) index, upper graph layers contain fewer nodes and support broad navigation, while lower layers contain denser local neighborhoods. Search starts from an entry point high in the hierarchy, greedily moves toward nodes closer to the query, descends through layers, and expands a candidate neighborhood at the base layer. The hierarchy supplies long-range movement before local refinement, rather than explicitly partitioning space into coarse cells. HNSW’s practical controls have clear conceptual roles. Greater graph connectivity stores more edges per node, often improving route quality but increasing index memory. More construction effort can choose better neighbors, at the cost of slower index builds. More query-time exploration visits additional nodes, generally improving ANN recall while increasing latency. The original HNSW work describes this layered proximity-graph construction and its high-recall search behavior [2].

Index family	Candidate mechanism	Key trade-offs
Flat exact	Score every vector.	Exact and simple, but query work and bandwidth grow with the corpus.
IVF	Probe selected coarse lists.	Probe count trades recall for scanned candidates; centroid and list balance can drift as the corpus changes.
HNSW	Navigate a layered proximity graph.	Edges and exploration trade index memory and search work for recall; deletion and compaction need explicit policy.
IVF + PQ	Probe lists and compare compressed codes.	Compression lowers storage and bandwidth but adds distance error; codebooks and vector revisions must remain aligned.

Table 1: IVF searches selected spatial regions, whereas HNSW navigates proximity edges. Neither dominates across corpus sizes, update patterns, recall targets, and hardware.

7 Candidate Generation, Filters, and Index Lifecycle

ANN is usually a first-stage candidate generator:

$$e_q \rightarrow \text{ANN} \rightarrow C(q) \rightarrow \text{optional \setminus reranking} \rightarrow R_{k(q)}. \quad (7)$$

The candidate set $C(q)$ should be large enough that a later, more discriminating stage has useful alternatives, but not so large that it violates latency or context-construction budgets. This chapter does not derive reranking; the important boundary is that an ANN miss cannot be repaired downstream if the required item never enters $C(q)$.

Real systems also apply structured constraints. Date, document type, language, tenant, access-control status, or product category may define which vectors are eligible before similarity ranking. **Pre-filtering** restricts the searchable population before or during ANN traversal; it preserves policy boundaries but can leave too few candidates when filters are selective. **Post-filtering** searches a broader set and discards ineligible results afterward; it can waste work and reduce the surviving top- k result count. The correct choice depends on filter selectivity, security semantics, index support, and whether an unauthorized vector is ever permitted to influence logs, caches, or a prompt.

Index build time and query time must be treated separately. Coarse centroids, PQ codebooks, and graph edges may require substantial offline construction, whereas query-time controls determine the work permitted per request. A mutable corpus adds insertion, deletion, and repair semantics. Removing a source might mean deleting its vector, marking it ineligible, rebuilding affected lists or graph neighborhoods, and invalidating cache entries. An index that reports a current document ID while retaining an old embedding, old metadata, or a deleted access policy is operationally stale even if it returns results quickly.

8 Memory, Hardware, and Vector Databases

Chapter 27’s *Nrb* payload estimate describes only raw embedding storage. Total vector-search memory may also include compressed codes, graph edges, coarse centroids, codebooks, IDs, metadata, filter structures, shard replicas, allocator overhead, and temporary search state. A graph can spend substantial memory on edges; an IVF design can consume additional memory for lists and centroids; a compressed index can save payload bytes while retaining original vectors for evaluation or reranking. Capacity planning must measure the whole index artifact rather than one vector matrix.

Hardware changes the apparent trade-offs. CPU search benefits from vector instructions, cache locality, and predictable memory access. GPU search can make batched flat scans and selected index operations efficient, but host-device transfer, device-memory capacity, and small request batches can dominate. Many ANN workloads are limited less by arithmetic throughput than by fetching scattered

vector, code, or graph data. Johnson, Douze, and Jegou demonstrate how similarity-search design can depend on GPU selection, memory hierarchy, and compressed-domain computation [3].

A **Vector Database** is an operational system that packages vector storage, ANN indexing, metadata, filtering, persistence, mutation, and often replication or distributed execution behind one interface. The name does not establish quality or correctness. It does not choose a compatible embedding model, guarantee ANN recall, solve access-control semantics, or determine whether retrieved text supports a generated claim. These remain explicit retrieval contracts regardless of whether the index is embedded in an application or exposed through a database service.

9 Failure Modes and Debugging

Low ANN recall can arise from too little IVF probing, insufficient graph exploration, poor cluster assignment, excessive compression, an unsuitable distance implementation, or a resource cap that truncates search. These are **index-quality** failures: exact search over the same embeddings may still retrieve strong candidates. By contrast, an **embedding-quality** failure persists under exact search because the desired item is not close under the learned score. A **downstream-ranking** failure occurs when the candidate set contains useful evidence but a later selector, context constructor, or generator discards or misuses it. This decomposition prevents an ANN parameter change from being misdiagnosed as an embedding-model improvement or degradation.

Other failures are operational. An index can use the wrong similarity metric or normalization convention, mix document vectors from different encoder revisions, return stale vectors after source updates, suffer memory blow-up from graph edges or replicas, or take too long to rebuild after corpus changes. Filters can silently eliminate the best candidates or, worse, be applied too late for a security boundary. A compression setting may look acceptable on average while collapsing rare entities, fine numerical distinctions, or close semantic alternatives. These risks require exact-baseline comparison, index-recall monitoring, corpus-version tracking, and task-level evaluation rather than a single latency number.

10 Implementation Contracts

The **metric contract** must bind the index to the query and document encoder revisions, embedding dimension, dtype, normalization state, score or distance convention, and tie rule. Tests should compare a small corpus against an exhaustive high-precision implementation and verify that the index's declared exact mode, if any, agrees with Equation 1.

The **index contract** must record the index family, build corpus revision, vector IDs, training sample for centroids or codebooks where applicable, coarse partition policy, compression configuration, graph construction policy, and all search controls. It must define the target ANN Recall@k and the latency evaluation protocol: query batch size, filter distribution, hardware, concurrency, warm-cache state, and percentile statistic. A single average latency without these conditions is not a portable index property.

The **mutation and policy contract** must state insertion, update, deletion, compaction, rebuild, and publication semantics. It must name the metadata and access-control filters applied before and after search, define how vectors excluded by policy are prevented from reaching returned candidates, and preserve the mapping from index result to source and chunk revision. Regression tests should separately measure exact-vector neighbors, ANN recall, task-relevance recall, filter correctness, and downstream ranking quality after every index or encoder revision.

11 Summary

Exact vector search supplies the reference candidate set but costs work proportional to the stored collection. ANN indexes reduce that work by narrowing the search or compressing the representation. IVF searches vectors from selected coarse regions; PQ stores compact approximate codes; HNSW navigates a hierarchy of proximity graphs. Their controls expose different combinations of build cost, memory, latency, and ANN recall.

ANN recall measures agreement with exact vector neighbors, not whether a RAG system retrieved relevant or truthful evidence. Reliable systems therefore separate embedding quality, index quality, filter behavior, and downstream ranking. The index must be versioned with the corpus, encoders, metric, policy filters, and mutation rules. Later chapters can build on its candidate set with lexical signals, reranking, query transformation, and evaluation, but none can restore evidence that candidate generation never supplied.

References

- [1] H. Jegou, M. Douze, and C. Schmid, “Product Quantization for Nearest Neighbor Search,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011, doi: 10.1109/TPAMI.2010.57.
- [2] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.
- [3] J. Johnson, M. Douze, and H. Jegou, “Billion-Scale Similarity Search with GPUs,” *arXiv preprint arXiv:1702.08734*, 2017, doi: 10.48550/arXiv.1702.08734.